



Customer Review Analysis

Minh Quang Duong
mduong@stevens.edu

Table of content

	Content	Page
I	Introduction (Problem we wanted to solve)	3
II	Data Description and Visualization (Datasets that we used)	3
III	Statistical Approach (Models we had tried)	6
IV	Result (How we thought the models would perform)	8
V	Evaluation (How we evaluate the performance the model on the dataset)	9
VI	AI as a teammate	9
VII	Conclusion	10
VIII	Reflection	11
IX	Appendix	12

I. Introduction:

In this digital age, anything and everything could be accomplished virtually. There are countless businesses that could help one finish certain tasks they want. The convenience and swiftness are specifically true when it comes to online shopping, and consequently product reviews as well. The extranet makes it so that there could be up to thousands of reviews for each product, each with its own context, rendering it nearly impossible to manually assess each one individually. One could count on the scores of the products for analysis, but it often only provides a surface-level overview. In order to address this gap, we set out to draw out the relationship between product reviews and score by analyzing the sentiment of the reviews by applying sentiment analysis.

Specifically, our goal is to build deep learning models that can learn from the text within the customer reviews and accurately predict the product's score. The process includes data cleaning, tokenization, vectorization, and training deep learning models, models testing and evaluation. This approach provides a deep understanding of product reviews beyond numerical scores.

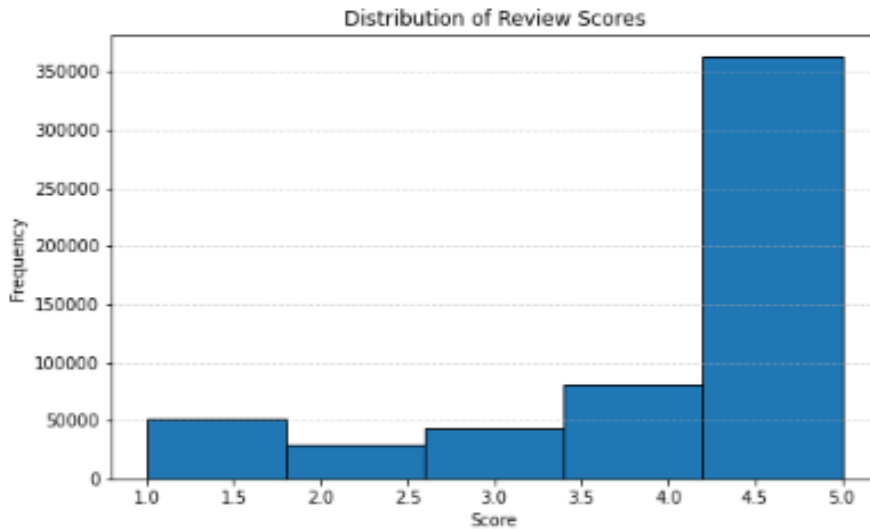
II. Data Description and Visualization:

The dataset is a set of Amazon Product Reviews on a wide range of different products. The original was shared by Arham Runi on:

[Amazon Product Reviews | Kaggle](#)

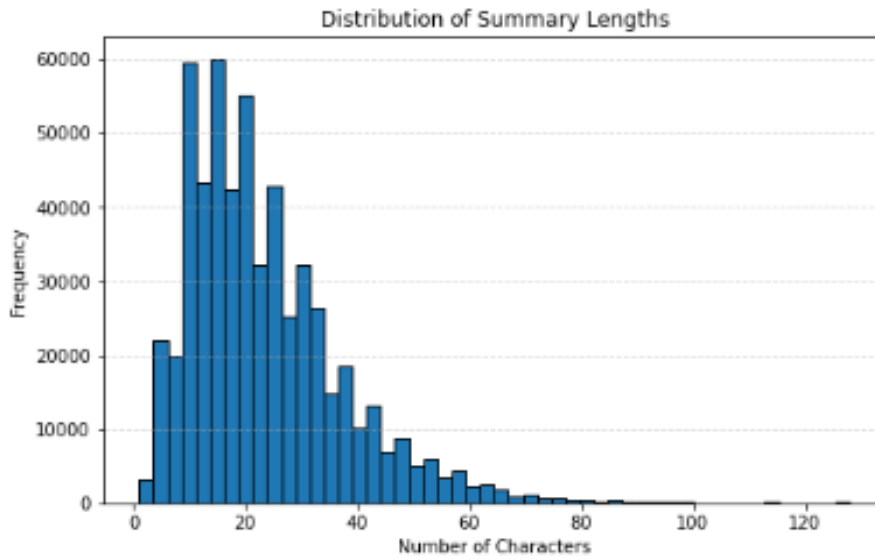
The dataset contains 568454 observations with 10 columns, namely: Id, ProductId, UserId, ProfileName, HelpfulnessNumerator, HelpfulnessDenominator, Score, Time, Summary, Text. For the purpose of our goal, we decided to focus only:

1. *Score* - Product Rating ranging from 1-5



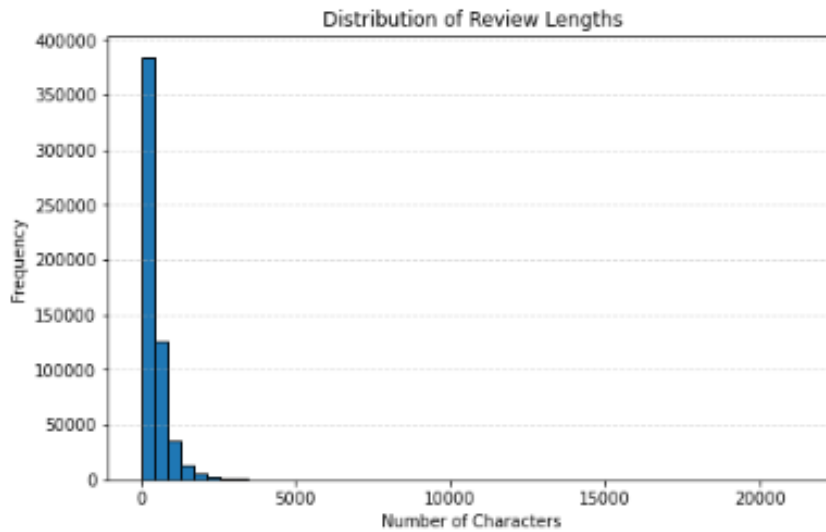
- The scores were heavily skewed toward 5. We implemented both binary classification and multi-class classification.
- For binary classification, we grouped score 1-4 together as 0 label and 5 as 1 label which made the distribution more balanced.
- For the multi-class classification, scores 1-5 were labeled 0-4 in order. Since the data was skewed towards 5, we employed custom weights for each label and also the focal loss function to ensure that the model will not ignore the minority and 'hard to predict' cases which in this scenario were scores 1-4.

2. *Summary* - The header or overview of the review



- The length average length of summary will help in determining the appropriate length for sequencing and padding.

3. *Text* - The full text content of the review



- The length average length of review will also help in determining the appropriate length for sequencing and padding.

The process of data preparation included the following:

1. *Data Cleaning:*
 - a. Converting text to lower case
 - b. Removing special characters
 - c. Removing Tags
 - d. Removing white noise
2. *Classification:*
 - a. Binary Classification
 - b. Multi-class Classification
3. *Tokenization:*
 - a. Tokenizing text
 - b. Converting text to sequence
 - c. Padding Sequences
4. *Data Split:*
 - a. Training Set: 80%
 - b. Validation Set: 10%
 - c. Testing Set: 10%

III. Statistical Approach:

As mentioned above, our goal is to build deep learning models that could accurately predict products' score given the context of its review. After weighing the benefits and drawbacks of different models, we have arrived at the selection the following three:

- a. Baseline Convolutional Neural Network (CNN)
 - i. *Advantage:*
 1. Swift training and inference
 2. Good for capturing n-gram patterns
 3. Less prone to overfitting

4. Lightweight

ii. *Disadvantage:*

1. Performs poorly at capturing long-range dependencies
2. Possible of struggle with complex sentiment structure

b. Baseline Long-Short Term Memory Model (LSTM)

i. *Advantage:*

1. Ability to capture long-range dependencies in text, making it possible for the model to fully grasp sentence/paragraph level sentiment.
2. Ability to model sequential data while keeping track of the order.
3. Handle contrast phrases and context-rich sentences better CNNs

ii. *Disadvantage:*

1. Slower training time.
2. More parameters which could lead to overfitting
3. Does not capture context from future words

c. Bidirectional Long-Short Term Memory Model (Bi-LSTM)

i. *Advantage:*

1. Process text in both forward and backward directions, capturing richer context
2. Higher effectiveness in understanding contrast cues and dependencies between long range dependencies
3. Outperform LSTMs on text classifications.

ii. *Disadvantage:*

1. Longer waiting time due to two pass of LSTM
2. Prone to overfitting without strong regularization or with small dataset.

IV. Result:

We expected that the CNN model would have the quickest time training and testing. LSTM and Bi-LSTM will take longer if not much longer but they will result in higher accuracy and while capturing longer dependencies between the text content.

a. Binary Classification: Refer to appendix A

Parameter/Model	CNN	LSTM	Bi - LSTM
Accuracy (Weighted)*	0.95	0.94	0.95
Precision (Weighted)	0.92	0.91	0.93
Recall (Weighted)	0.92	0.92	0.91
F1-Score (Weighted)*	0.92	0.92	0.92
Estimated Avg Training Time	2:00 / epoch	8:00 / epoch	13:00 / epoch

b. Multi-class Classification: Refer to appendix B

Parameter/Model	CNN	LSTM	Bi - LSTM
Accuracy (Weighted)*	0.79	0.79	0.78
Precision (Weighted)	0.77	0.77	0.76
Recall (Weighted)	0.79	0.79	0.78
F1-Score (Weighted)*	0.77	0.79	0.76
Estimated Avg Training Time	2:30 / epoch	8:30 / epoch	13:45 / epoch

V. Evaluation:

Parameter/Model	CNN	LSTM	Bi - LSTM
Accuracy	Relatively similar accuracy in both case of classification with the difference being in the 0.01s		
F1-Score	1st in Binary 2nd in Multi-class	1st in Binary 1st in Multi-class	1st in Binary 3rd in Multi-class
Dependency	Capture local patterns only	Capture sequential data and temporal dependencies	Capture both past and future context of sequence
Training Time (Average)	Quickest	3x CNN	almost 2x LSTM

VI. AI as teammates:

Throughout the whole project our AI teammates have contributed greatly to the process of building and analyzing our models. To be specific, the areas in which they provided the most help were model evaluation and parameter tuning. As these models were trained and applied to the testing data, we realized that there were changes that needed to be made:

1. Binary Classification and Multi-class Classification:

- a. We came to the conclusion that simply categorizing the score as positive and negative would not provide the whole context of the review, hence our AI teammate

had suggested a multi-class classification in comparison.

2. Parameters Tuning and Focal Loss Application

- a. The models were taking more time than expected to train, we suspected batch size to be the main culprit but our AI teammate was able to identify another key indicator which was the sequencing length. Since our approach was to combine the summary and main reviews together, it will take a shorter length to capture the context. Since we reduced the fixed length to quick, all sentences were subjected after tokenization. (Refer to Appendix A for the result post-tuning and pre-focal loss)
- b. The application of focal loss after tuning batch size and sequencing length was also a suggestion from our AI teammate in order to combat the imbalance in distribution of score. (Refer to Appendix B for the result pre-focal loss)

VII. Conclusion:

As shown in the comparison of the models' testing results, there seems to be very similar performance between the models in both cases of classification. Instead of declaring one model as the clear best in this case, we have come to the conclusion that the application of all three models is needed throughout the analysis and classification process. CNN can be used to capture local patterns which is useful for short reviews. For lengthy reviews with long dependencies, it's clear that

LSTM will perform the best as it was designed for this purpose, also it has the best F1-score and accuracy out of the models. Bi-LSTM will be very useful for reviews of extreme length, as it will be able to capture the richer context offered by the immense depth of the reviews.

VIII. Reflection:

1. Application of three-model pipeline:
 - a. As mentioned above, we would continue the analysis by building a pipeline that will process the data with the appropriate model. Short reviews will be processed by CNN, lengthy reviews will be processed by LSTM and once the length reaches a certain threshold, all reviews will then be processed by Bi-LSTM.
2. Application of a different model:
 - a. We would also want to apply the model that is not a neural network and that is the BERT model (Bidirectional Encoder Representations from Transformers).

IX. Appendix:

A. Binary Classification with batch size 256, MAX_LEN 100

CNN:

Test Loss: 0.1482
Test Accuracy: 0.9469
Test Precision: 0.9666
Test Recall: 0.9654
1777/1777  5s 3ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.88	0.88	12468
1	0.97	0.97	0.97	44378
accuracy			0.95	56846
macro avg	0.92	0.92	0.92	56846
weighted avg	0.95	0.95	0.95	56846

Confusion Matrix:
[[10987 1481]
[1535 42843]]

LSTM:

Test Loss: 0.1509
Test Accuracy: 0.9430
Test Precision: 0.9659
Test Recall: 0.9609
1777/1777  37s 20ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.88	0.87	12468
1	0.97	0.96	0.96	44378
accuracy			0.94	56846
macro avg	0.91	0.92	0.92	56846
weighted avg	0.94	0.94	0.94	56846

Confusion Matrix:
[[10961 1507]
[1735 42643]]

Bi-LSTM:

Test Loss: 0.1416
 Test Accuracy: 0.9475
 Test Precision: 0.9573
 Test Recall: 0.9762
 1777/1777 ————— 76s 43ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.85	0.88	12468
1	0.96	0.98	0.97	44378
accuracy			0.95	56846
macro avg	0.93	0.91	0.92	56846
weighted avg	0.95	0.95	0.95	56846

Confusion Matrix:
 [[10537 1931]
 [1056 43322]]

B. Multi-class Classification with batch size 256, MAX_LEN 100 and with focal loss + weights

CNN:

Test Loss: 0.0645
 Test Accuracy: 0.7904
 1777/1777 ————— 7s 4ms/step

Test Precision: 0.7674
 Test Recall: 0.7904

Classification Report:

	precision	recall	f1-score	support
0	0.69	0.84	0.76	5227
1	0.52	0.30	0.38	2977
2	0.58	0.51	0.54	4264
3	0.61	0.32	0.42	8066
4	0.86	0.96	0.91	36312
accuracy			0.79	56846
macro avg	0.65	0.59	0.60	56846
weighted avg	0.77	0.79	0.77	56846

Confusion Matrix:
 [[4381 298 180 19 349]
 [1184 898 589 62 244]
 [422 436 2184 626 596]
 [105 60 614 2561 4726]
 [277 23 200 906 34906]]

LSTM:

1777/1777 ————— 44s 24ms/step

Test Loss: 0.0639
Test Accuracy: 0.7877
Test Precision: 0.7664
Test Recall: 0.7877

Classification Report:

	precision	recall	f1-score	support
0	0.68	0.86	0.76	5227
1	0.54	0.21	0.30	2977
2	0.54	0.49	0.51	4264
3	0.56	0.39	0.46	8066
4	0.87	0.95	0.91	36312
accuracy			0.79	56846
macro avg	0.64	0.58	0.59	56846
weighted avg	0.77	0.79	0.77	56846

Confusion Matrix:

```
[[ 4501  195  239   29  263]
 [ 1310  616  773   80  198]
 [  458  297 2079   973  457]
 [  108   29  546 3179 4204]
 [   272   13  223 1401 34403]]
```

Bi-LSTM:

1777/1777 ————— 63s 36ms/step

Test Loss: 0.0623
Test Accuracy: 0.7847
Test Precision: 0.7599
Test Recall: 0.7847

Classification Report:

	precision	recall	f1-score	support
0	0.68	0.84	0.75	5227
1	0.48	0.33	0.39	2977
2	0.55	0.46	0.50	4264
3	0.60	0.28	0.38	8066
4	0.85	0.96	0.91	36312
accuracy			0.78	56846
macro avg	0.63	0.58	0.59	56846
weighted avg	0.76	0.78	0.76	56846

Confusion Matrix:

```
[[ 4389  402  147   12  277]
 [ 1177  993  539   39  229]
 [  456  610 1969   628  601]
 [  135   44  719 2262 4906]
 [   300   28  194   798 34992]]
```

C. Binary and Multi-class F1-score chart:

